

Plano De Execucao Atualizado: SaaS Arc Flash Com PDF Do Plano E

Controle De Uso Codex

Summary

Criar um MVP comercial web para calculo de arc flash, começando em VM Linux local com Docker, depois staging com Supabase Free e producao/beta com Supabase Pro.

Antes da implementacao tecnica, gerar dentro do repositorio:

- * 'docs/planejamento/arcflash-mvp-plano.md'
- * 'docs/planejamento/arcflash-mvp-plano.pdf'
- * 'docs/planejamento/codex-usage-log.md'

A ferramenta sera apoio tecnico para estudos de arc flash, com validacao final por engenheiro habilitado.

Fase 0: Preparacao Do Ambiente Local

Ambiente atual:

- * VirtualBox Linux.
- * 4 vCPUs.
- * 16 GB RAM.
- * 60 GB disco.
- * VS Code, Docker, Portainer e Codex Plus configurados.

Primeiras tarefas da execucao:

- * Criar estrutura de documentacao do projeto.
- * Salvar este plano em Markdown.
- * Gerar PDF A4 simples do plano, preferencialmente via container com 'pandoc' ou ferramenta equivalente, sem instalar dependencias globais desnecessarias.
- * Criar '.gitignore' protegendo:
 - * 'referencias/normas-local/'
 - * arquivos '.env'
 - * dumps locais
 - * volumes e caches.
- * Criar 'docs/planejamento/codex-usage-log.md' para controle manual por sessao.

Controle De Uso Do Codex Plus

A OpenAI informa que os limites do Codex variam por plano, tamanho da tarefa, complexidade, duracao e contexto usado. Tambem ha limites por janela de 5 horas e limite semanal compartilhado entre tarefas locais e cloud tasks.

Fonte: <https://help.openai.com/en/articles/11369540>

Como nao ha garantia de um contador local/API simples para a assinatura ChatGPT Pro, o MVP do controle sera operacional:

- * Registrar cada sessao Codex em 'docs/planejamento/codex-usage-log.md'.
- * Campos:
- * data/hora;
- * objetivo da sessao;
- * arquivos tocados;
- * modelo usado, se visivel;
- * duracao;
- * tipo de tarefa: planejamento, codigo, teste, revisao;
- * resultado;
- * observacao de limite, se aparecer aviso na interface.
- * Trabalhar com blocos pequenos para reduzir contexto.
- * Preferir tarefas locais no Codex para alteracoes pequenas.
- * Usar subagentes apenas quando houver tarefas independentes.
- * Evitar reenviar normas, PDFs, logs grandes ou arquivos irrelevantes.
- * Fazer checkpoints: no maximo 1 bloco grande por sessao antes de revisar.

Fase 1: Infra Local Docker

- * Criar Docker Compose com:
- * 'app': Next.js/TypeScript.
- * 'db': Postgres.
- * 'storage': MinIO ou storage local.
- * 'mail': Mailpit.
- * 'worker': PDF, uploads e extracao assistida.
- * Criar scripts:
- * 'make dev'
- * 'make test'
- * 'make db-migrate'
- * 'make db-reset'
- * 'make logs'
- * 'make docker-clean'
- * Usar Postgres container no inicio.
- * Introduzir Supabase local quando Auth, Storage e RLS forem implementados.

Fase 2: Staging Supabase Free

- * Usar Supabase Free apenas para staging.
- * Testar auth, RLS, storage, upload, download, relatorios e webhooks.
- * Nao usar para clientes pagantes nem dados sensiveis reais.

Fase 3: Beta/Producao Supabase Pro

- * Migrar para Supabase Pro antes de usuarios externos reais.
- * Configurar dominio, backups, logs, monitoramento e chaves de producao.
- * Usar Mercado Pago para cobranca.
- * Usar OpenAI API apenas no recurso premium de extracao assistida de curvas.

Blocos De Produto

- * Fundacao SaaS: login, usuarios, organizacoes, permissoes e dashboard.
- * Modelo de dados: projetos, equipamentos, pontos de analise, curvas, anexos e relatorios.

- * Motor de calculo: biblioteca isolada, deterministica, versionada e testada.
- * Estudos: formularios guiados, entrada manual de curvas e validacoes.
- * Anexos: upload de imagens/PDFs, preservados na documentacao do projeto.
- * IA premium: extracao assistida de curvas com confirmacao manual obrigatoria.
- * Relatorios: PDF com memorial, entradas, resultados, anexos, versao do calculo e termo de validacao.
- * Comercializacao: planos teste, profissional e premium, com limites por projeto, storage, relatorio e IA.

Test Plan

- * Confirmar que o plano em Markdown e PDF foi gerado.
- * Confirmar que arquivos sensiveis/normas locais estao ignorados pelo Git.
- * Confirmar que Docker Compose sobe os servicos locais.
- * Confirmar migrations em Postgres local e Supabase.
- * Testar Auth, RLS e storage.
- * Testar motor de calculo com casos aprovados por engenheiro.
- * Testar geracao de relatorio PDF.
- * Testar limites por plano.
- * Testar extracao IA com confirmacao manual.

Assumptions

- * PDF inicial sera simples, A4, sem identidade visual definitiva.
- * Controle de uso Codex sera manual/operacional no inicio, porque os limites oficiais variam conforme uso e contexto.
- * Supabase Free sera staging, nao producao.
- * Supabase Pro sera usado antes de clientes pagantes.
- * O disco de 60 GB sera monitorado; ideal ampliar para 100-120 GB se o volume de Docker/uploads crescer.